
Node.js

Interview Questions & Answers | 24 questions | Server-side JavaScript runtime

Contents

Node.js Fundamentals (Q1-Q5)

Modules, npm & Package Management (Q6-Q9)

Asynchronous Patterns & Streams (Q10-Q13)

Building Servers & APIs (Q14-Q18)

Performance, Debugging & Best Practices (Q19-Q24)

Node.js Fundamentals

Q1. What is Node.js and how is it different from a browser JavaScript runtime?

Node.js is a JavaScript runtime built on Chrome's V8 engine that lets JS run outside the browser. It adds APIs for file systems, networking, processes, and modules that don't exist in the browser, while removing browser-only globals like `window` and `document`. It's commonly used to build servers, CLIs, and build tooling.

Q2. Explain the Node.js event loop and its phases.

Node's event loop (implemented via libuv) processes callbacks in distinct phases each iteration.

- **Timers:** executes callbacks scheduled by `setTimeout/setInterval`
- **Pending callbacks:** executes I/O callbacks deferred from the previous cycle
- **Poll:** retrieves new I/O events and executes their callbacks
- **Check:** executes `setImmediate()` callbacks
- **Close callbacks:** executes e.g. `socket.on('close', ...)`

`process.nextTick()` and Promise microtasks run between every phase, before the loop continues.

Q3. Is Node.js single-threaded? How does it handle concurrency?

The main JavaScript execution thread is single-threaded, but Node offloads expensive or blocking work (file I/O, DNS, some crypto) to a libuv thread pool (default 4 threads) behind the scenes. This lets Node handle thousands of concurrent I/O operations without blocking the event loop, though CPU-bound JS code still runs synchronously on the single thread.

Q4. What is the difference between `process.nextTick()` and `setImmediate()`?

`process.nextTick()` queues a callback to run immediately after the current operation completes, before the event loop continues to the next phase - it has higher priority than promise microtasks even. `setImmediate()` queues a callback to run in the 'check' phase, after the current poll phase completes.

Q5. What are the core modules in Node.js?

Core modules ship with Node and don't require installation.

- `fs` - file system operations
- `http/https` - creating servers and making requests
- `path` - cross-platform file path utilities
- `events` - the `EventEmitter` class
- `stream` - handling streaming data
- `crypto` - hashing, encryption, and randomness
- `os` - operating system information
- `child_process` / `cluster` - spawning processes

Modules, npm & Package Management

Q6. What is the difference between CommonJS and ES Modules in Node.js?

CommonJS (require/module.exports) is Node's original, synchronous module system. ES Modules (import/export) are the standardized, asynchronous, statically analyzable format. Node supports both: use a .mjs extension or set "type": "module" in package.json to opt into ESM, or .cjs to force CommonJS.

Q7. What is the difference between dependencies, devDependencies, and peerDependencies in package.json?

dependencies are required at runtime in production. devDependencies are only needed for local development and testing (linters, test runners, build tools) and are excluded with npm install --production. peerDependencies declare a package the consumer's project must provide itself, typically used by plugins/libraries that expect a specific version of a framework to already be installed.

Q8. What is the purpose of package-lock.json?

package-lock.json records the exact resolved version of every installed package (including transitive dependencies), ensuring that npm install produces an identical dependency tree across machines and over time, regardless of caret/tilde ranges in package.json.

Q9. How does Node.js resolve a 'require' call?

Node checks, in order: core modules, then relative/absolute paths (./ or /), then walks up node_modules directories from the current folder to the filesystem root looking for a matching package. It also caches modules after the first load, so subsequent requires return the same cached instance.

Asynchronous Patterns & Streams

Q10. What is the EventEmitter and how is it used?

EventEmitter is a core class implementing the observer pattern - objects can emit named events and other code can subscribe with .on(). It underlies most of Node's async APIs, including streams and HTTP servers.

```
const EventEmitter = require('events');
class Ticker extends EventEmitter {}
const ticker = new Ticker();
ticker.on('tick', (count) => console.log('Tick', count));
ticker.emit('tick', 1);
```

Q11. What are Streams in Node.js and what types exist?

Streams process data piece by piece (chunks) instead of loading it all into memory at once, which is essential for handling large files or network data efficiently.

- Readable - a source of data you can read from (e.g. fs.createReadStream)
- Writable - a destination you can write data to (e.g. fs.createWriteStream)
- Duplex - both readable and writable (e.g. a TCP socket)
- Transform - a duplex stream that modifies data as it passes through (e.g. zlib)

```
const fs = require('fs');
fs.createReadStream('big.txt')
  .pipe(fs.createWriteStream('copy.txt'));
```

Q12. How do you handle errors in async/await code in Node?

Wrap awaited calls in try/catch blocks, or attach a .catch() handler on the returned promise. For uncaught cases, listen to process.on('unhandledRejection') and process.on('uncaughtException') as a last line of defense, but the goal should be to handle errors close to where they occur, not to rely on these global handlers.

Q13. What is backpressure in the context of streams?

Backpressure happens when a writable stream can't process incoming data as fast as a readable stream produces it. `write()` returns `false` when its internal buffer is full, signaling the source to pause until a 'drain' event fires. The `.pipe()` method handles this automatically, which is why it's preferred over manually reading and writing chunks.

Building Servers & APIs

Q14. How do you create a basic HTTP server in Node.js without a framework?

Using the built-in `http` module.

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ message: 'Hello' }));
});
server.listen(3000, () => console.log('Listening on 3000'));
```

Q15. What is middleware in Express.js?

Middleware are functions that run in the request-response cycle with access to `req`, `res`, and a `next()` function. They can modify the request/response, end the cycle, or call `next()` to pass control to the next middleware. Used for logging, auth, parsing bodies, error handling, etc.

```
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});
```

Q16. How would you handle centralized error handling in an Express app?

Define an error-handling middleware with four parameters (`err`, `req`, `res`, `next`), placed after all routes. Pass errors to it via `next(err)` from any route or middleware, or use async wrapper utilities so rejected promises are forwarded automatically.

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({ error: err.message });
});
```

Q17. What is CORS and how do you handle it in Node?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that restricts web pages from making requests to a different origin than the one that served the page, unless the server explicitly allows it via response headers like `Access-Control-Allow-Origin`. In Express, this is typically handled with the `'cors'` middleware package.

Q18. How do you validate and sanitize incoming request data in a Node API?

Use a schema validation library (Joi, Zod, `express-validator`) at the boundary of your API to reject malformed input early, before it reaches business logic, and to strip unexpected fields. This prevents injection attacks and downstream type errors.

Performance, Debugging & Best Practices

Q19. How would you debug a memory leak in a Node.js application?

Take heap snapshots at intervals using the built-in inspector (`node --inspect`) or Chrome DevTools, compare them to spot growing object counts, and look for common culprits: unbounded caches, closures holding references, event listeners that are never removed, and global arrays that keep growing. Tools like `clinic.js` or `heapdump` can automate the process.

Q20. What is clustering in Node.js and why use it?

Since Node runs JS on a single thread, the `cluster` module lets you fork multiple worker processes (typically one per CPU core) that share the same server port, letting a Node app use all available cores and improving throughput and resilience - if one worker crashes, others keep serving.

Q21. What is the difference between clustering and `worker_threads`?

`cluster` forks full child processes (separate memory, separate event loops) for scaling network-bound work like HTTP servers. `worker_threads` run true multi-threaded JavaScript within the same process with optional shared memory (`SharedArrayBuffer`), better suited for CPU-intensive tasks like image processing or heavy computation.

Q22. How do environment variables and configuration management typically work in Node apps?

Environment-specific config (API keys, DB URLs, ports) is read from `process.env`, often loaded from a `.env` file in development via a package like `dotenv`, and injected directly by the hosting platform in production. Secrets should never be committed to source control.

Q23. What are some common Node.js security best practices?

Security is an ongoing concern across the whole stack.

- Keep dependencies updated and audit them (`npm audit`)
- Validate and sanitize all user input
- Use parameterized queries to prevent SQL/NoSQL injection
- Set security headers (`helmet` middleware) and enforce HTTPS
- Avoid `eval()` and `dynamic require()` with user input
- Rate-limit APIs and set sane request body size limits
- Store secrets in environment variables or a secrets manager, never in code

Q24. How does Node.js handle uncaught exceptions and unhandled promise rejections?

An uncaught synchronous exception crashes the process by default unless caught by a `process.on('uncaughtException')` handler. An unhandled promise rejection triggers a warning (and in modern Node versions, also crashes the process by default). Best practice is to handle errors locally and use these global handlers only for logging before a graceful shutdown, not as normal control flow.